

DSA — Data Structures & Algorithms

Java Backend Interview — Short Notes

1. Big-O Complexity Reference

Complexity	Name	Example	N=100	N=10K
O(1)	Constant	HashMap get, array index	1	1
O(log N)	Logarithmic	Binary search, balanced BST	7	14
O(N)	Linear	Linear scan, single loop	100	10K
O(N log N)	Linearithmic	Merge sort, heap sort, TreeMap ops	700	140K
O(N²)	Quadratic	Bubble sort, nested loops	10K	100M
O(2^N)	Exponential	All subsets, recursive fibonacci	huge	inf
O(N!)	Factorial	All permutations	inf	inf

✓ Target O(N log N) or better for interviews. O(N²) may be acceptable for small N (under 1000) with justification.

2. Arrays & Strings

2.1 Two Pointers

- Use when: sorted array, find pair/subarray, shrink search space from both ends.

```
// Two sum in sorted array – O(N) time, O(1) space
int left=0, right=arr.length-1;
while (left < right) {
    int sum = arr[left] + arr[right];
    if (sum == target) return new int[]{left, right};
    else if (sum < target) left++;
    else right--;
}
```

2.2 Sliding Window

- Use when: contiguous subarray/substring, max/min window satisfying condition.

```
// Longest substring without repeating chars – O(N)
Map<Character,Integer> map = new HashMap<>();
int max=0, left=0;
for (int right=0; right < s.length(); right++) {
    char c = s.charAt(right);
    if (map.containsKey(c)) left = Math.max(left, map.get(c)+1);
    map.put(c, right);
    max = Math.max(max, right-left+1);
}
```

2.3 Prefix Sum

- Use when: range sum queries, count subarrays with sum = k.

```
// Subarray sum equals k – O(N)
Map<Integer,Integer> freq = new HashMap<>(); freq.put(0,1);
int sum=0, count=0;
```

```

for (int n : nums) {
    sum += n;
    count += freq.getOrDefault(sum-k, 0);
    freq.merge(sum, 1, Integer::sum);
}

```

3. Sorting Algorithms

Algorithm	Best	Avg	Worst	Space	Stable	Notes
Bubble Sort	$O(N)$	$O(N^2)$	$O(N^2)$	$O(1)$	Yes	Simple. Swap adjacent. Early exit if sorted.
Insertion Sort	$O(N)$	$O(N^2)$	$O(N^2)$	$O(1)$	Yes	Good for small or nearly-sorted arrays.
Merge Sort	$O(N \log N)$	$O(N \log N)$	$O(N \log N)$	$O(N)$	Yes	Stable, guaranteed. Good for linked lists.
Quick Sort	$O(N \log N)$	$O(N \log N)$	$O(N^2)$	$O(\log N)$	No	Best avg. Random pivot avoids worst case.
Heap Sort	$O(N \log N)$	$O(N \log N)$	$O(N \log N)$	$O(1)$	No	In-place, guaranteed $N \log N$. Not cache-friendly.
Tim Sort	$O(N)$	$O(N \log N)$	$O(N \log N)$	$O(N)$	Yes	Java <code>Arrays.sort(Object[])</code> — hybrid merge+insertion.

- Java `Arrays.sort(int[])` = dual-pivot QuickSort $O(1)$ extra. `Arrays.sort(Object[])` = TimSort (stable).

4. Linked List Patterns

4.1 Fast & Slow Pointers (Floyd's)

- Detect cycle, find middle, find Nth from end.

```

// Detect cycle –  $O(N)$  time  $O(1)$  space
ListNode slow=head, fast=head;
while (fast!=null && fast.next!=null) {
    slow=slow.next; fast=fast.next.next;
    if (slow==fast) return true; // cycle
}
return false;

// Find middle – slow stops at middle when fast reaches end
ListNode slow=head, fast=head;
while (fast!=null && fast.next!=null) {
    slow=slow.next; fast=fast.next.next;
}
return slow; // middle node

```

4.2 Reverse Linked List — Iterative $O(N)$

```

ListNode prev=null, curr=head;
while (curr!=null) {
    ListNode next=curr.next;
    curr.next=prev;
    prev=curr; curr=next;
}
return prev; // new head

```

5. Trees

5.1 Tree Traversals

Traversal	Order	Use Case	Stack Approach
Inorder	Left, Root, Right	BST gives sorted order	Morris or explicit stack
Preorder	Root, Left, Right	Copy tree, serialize	Push right then left to stack
Postorder	Left, Right, Root	Delete tree, expression eval	Reverse of modified preorder
Level Order	Level by level (BFS)	Shortest path, level averages	Queue — add children of dequeued node

```
// BFS Level Order — O(N)
Queue<TreeNode> q = new LinkedList<>(); q.offer(root);
List<List<Integer>> result = new ArrayList<>();
while (!q.isEmpty()) {
    int size = q.size();
    List<Integer> level = new ArrayList<>();
    for (int i=0; i < size; i++) {
        TreeNode node = q.poll();
        level.add(node.val);
        if (node.left!=null) q.offer(node.left);
        if (node.right!=null) q.offer(node.right);
    }
    result.add(level);
}
```

5.2 BST & Trie

- BST: left < root < right. Inorder = sorted. $O(\log N)$ avg for search/insert/delete. Worst $O(N)$ unbalanced.
- Java TreeMap/TreeSet = Red-Black BST — $O(\log N)$ guaranteed.
- Trie: autocomplete, spell check. Insert/search $O(L)$ where L = word length. TrieNode has children[26] + isEnd flag.

6. Graphs

6.1 Representations

Representation	Space	Check Edge	List Neighbors	Best For
Adjacency List	$O(V+E)$	$O(\text{degree})$	$O(\text{degree})$	Sparse graphs — most practical problems
Adjacency Matrix	$O(V^2)$	$O(1)$	$O(V)$	Dense graphs, quick edge existence check
Edge List	$O(E)$	$O(E)$	$O(E)$	Kruskal's MST, simple representation

6.2 BFS vs DFS

	BFS	DFS
Data structure	Queue	Stack / recursion
Time / Space	$O(V+E)$ / $O(V)$	$O(V+E)$ / $O(V)$ recursion depth
Shortest path	Yes — unweighted graph	No
Use for	Shortest path, level order, bipartite check, connectivity	Cycle detection, topological sort, DFS backtracking, path finding

6.3 Topological Sort — Kahn's Algorithm

```
int[] inDegree = new int[n];
for (int[] edge : edges) inDegree[edge[1]]++;
Queue<Integer> q = new LinkedList<>();
for (int i=0; i < n; i++) if (inDegree[i]==0) q.offer(i);
List<Integer> order = new ArrayList<>();
while (!q.isEmpty()) {
    int node = q.poll(); order.add(node);
    for (int nei : adj.get(node)) if (--inDegree[nei]==0) q.offer(nei);
}
return order.size()==n ? order : new ArrayList<>(); // empty = cycle exists
```

6.4 Shortest Path Algorithms

Algorithm	Weights	Time	Use When
BFS	Unweighted	$O(V+E)$	Minimum hops in unweighted graph
Dijkstra	Non-negative	$O((V+E) \log V)$	Weighted, single source, no negative edges
Bellman-Ford	Negative OK	$O(VE)$	Negative edges, detect negative cycles
Floyd-Warshall	All pairs	$O(V^3)$	All-pairs shortest path, dense graph

7. Dynamic Programming

- When to use: overlapping subproblems + optimal substructure.
- Approach: (1) Identify state. (2) Write recurrence. (3) Memoization or tabulation. (4) Space optimize.

7.1 Classic DP Patterns

Problem	State	Recurrence	Pattern
Fibonacci	$dp[i]$	$dp[i] = dp[i-1] + dp[i-2]$	1D DP
Coin Change	$dp[amount]$	$dp[i] = \min(dp[i-coin]+1)$ for each coin	1D — unbounded knapsack
0/1 Knapsack	$dp[i][w]$	$\max(\text{exclude: } dp[i-1][w], \text{include: } dp[i-1][w-wt]+val)$	2D include/exclude
LCS	$dp[i][j]$	match: $dp[i-1][j-1]+1$; else $\max(dp[i-1][j], dp[i][j-1])$	2D two sequences
LIS	$dp[i]$	$dp[i] = \max(dp[j]+1)$ for $j < i$ where $arr[j] < arr[i]$	$O(N^2)$ or $O(N \log N)$
Edit Distance	$dp[i][j]$	match: $dp[i-1][j-1]$; else $1 + \min(\text{insert, delete, replace})$	2D two strings

```
// Coin change — O(amount * coins) time, O(amount) space
int[] dp = new int[amount+1];
Arrays.fill(dp, Integer.MAX_VALUE); dp[0]=0;
for (int i=1; i <= amount; i++)
    for (int coin : coins)
        if (coin <= i && dp[i-coin] != Integer.MAX_VALUE)
            dp[i] = Math.min(dp[i], dp[i-coin]+1);
return dp[amount] == Integer.MAX_VALUE ? -1 : dp[amount];
```

8. Heap & Priority Queue

- Min-heap: `PriorityQueue<Integer>()`. Max-heap: `PriorityQueue<>(Comparator.reverseOrder())`.
- **Top K elements:** min-heap of size K — $O(N \log K)$. Keep ejecting smallest.

- **Merge K sorted lists:** min-heap of (value, listIndex) — $O(N \log K)$.
- **Median of data stream:** max-heap (lower half) + min-heap (upper half). Balance sizes.

```
// Top K frequent elements –  $O(N \log K)$ 
Map<Integer,Integer> freq = new HashMap<>();
for (int n : nums) freq.merge(n, 1, Integer::sum);
PriorityQueue<int[]> pq = new PriorityQueue<>((a,b) -> a[1]-b[1]);
for (var e : freq.entrySet()) {
    pq.offer(new int[]{e.getKey(), e.getValue()});
    if (pq.size() > k) pq.poll();
}
```

9. Backtracking

- Try choices recursively. Undo (backtrack) on dead end. Pattern: choose → recurse → unchoose.
- Use for: permutations, combinations, subsets, N-Queens, Sudoku, word search.

```
// Generate all subsets –  $O(2^N)$ 
void backtrack(int[] nums, int start, List<Integer> curr, List<List<Integer>> res) {
    res.add(new ArrayList<>(curr)); // add at every node
    for (int i=start; i < nums.length; i++) {
        curr.add(nums[i]); // choose
        backtrack(nums, i+1, curr, res); // recurse
        curr.remove(curr.size()-1); // unchoose
    }
}
```

10. Binary Search Patterns

- Use when: sorted or monotonic array. $O(\log N)$. Key: avoid int overflow with $\text{mid} = \text{lo} + (\text{hi} - \text{lo}) / 2$.

```
// Classic – find target in sorted array
int lo=0, hi=arr.length-1;
while (lo <= hi) {
    int mid = lo + (hi-lo)/2;
    if (arr[mid] == target) return mid;
    else if (arr[mid] < target) lo = mid+1;
    else hi = mid-1;
}
return -1;

// Lower bound – leftmost index where arr[i] >= target
int lo=0, hi=arr.length;
while (lo < hi) {
    int mid = lo + (hi-lo)/2;
    if (arr[mid] < target) lo = mid+1;
    else hi = mid; // don't return – keep shrinking right
}
return lo;
```

11. Java Collections Complexity Reference

Collection	Add	Remove	Get/Search	Space	Notes
ArrayList	$O(1)$ amort	$O(N)$	$O(1)$ / $O(N)$	$O(N)$	Dynamic array. 1.5x grow. remove(i) shifts elements.

LinkedList	$O(1)$	$O(1)^*$	$O(N)$ both	$O(N)$	* $O(1)$ with iterator. Random access $O(N)$.
ArrayDeque	$O(1)$ amort	$O(1)$	$O(1)$ ends	$O(N)$	Prefer over Stack/LinkedList for queue/deque.
HashMap	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg	$O(N)$	$O(N)$ worst (hash collision). Load factor 0.75.
TreeMap/Set	$O(\log N)$	$O(\log N)$	$O(\log N)$	$O(N)$	Sorted. Red-Black BST. NavigableMap/Set ops.
PriorityQueue	$O(\log N)$	$O(\log N)$	$O(N)$ search	$O(N)$	Min-heap. $\text{poll()}=O(\log N)$. $\text{peek()}=O(1)$.

12. Interview Questions

Arrays & Strings

Q1. Find maximum sum subarray (Kadane's)?

→ Maintain currentMax and globalMax. At each element: $\text{currentMax} = \max(\text{num}, \text{currentMax} + \text{num})$. If $\text{currentMax} > \text{globalMax}$ update it. $O(N)$ time, $O(1)$ space. Reset to num if starting fresh beats extending.

Q2. Two sum — multiple approaches?

→ (1) Brute force $O(N^2)$: nested loop. (2) HashMap $O(N)$: for each num check if complement ($\text{target} - \text{num}$) exists in map, else add. (3) Sorted array + two pointers $O(N \log N)$: sort then converge from both ends.

Q3. How to detect anagram?

→ Sort both strings — $O(N \log N)$. Or frequency count array[26] — $O(N)$. For grouping anagrams: HashMap keyed by sorted string, value is list of original strings.

Linked List & Trees

Q4. Detect and find cycle start in linked list?

→ Phase 1: fast and slow pointers meet inside cycle. Phase 2: reset one pointer to head, advance both at speed 1. They meet at cycle start. $O(N)$ time, $O(1)$ space — Floyd's algorithm.

Q5. Lowest Common Ancestor of BST?

→ If both $p, q < \text{root}$: go left. If both $> \text{root}$: go right. Otherwise root is LCA. $O(H) = O(\log N)$ balanced. For binary tree (not BST): recurse — if $\text{root} == p$ or $\text{root} == q$ return root; check left and right; if both non-null, root is LCA.

Q6. Validate BST?

→ Inorder traversal — verify strictly increasing. Or pass bounds: $\text{isValid}(\text{node}, \text{min}, \text{max})$. Each node must satisfy $\text{min} < \text{node.val} < \text{max}$. $O(N)$.

Sorting & Searching

Q7. QuickSort vs MergeSort — when to use?

→ QuickSort: in-place $O(1)$ space, best avg $O(N \log N)$, unstable, $O(N^2)$ worst. Good for arrays — Java primitive arrays use dual-pivot QS. MergeSort: $O(N)$ space, always $O(N \log N)$, stable. Good for linked lists, external sort, when stability needed.

Q8. Find Kth largest in unsorted array?

→ QuickSelect: avg $O(N)$, worst $O(N^2)$. Min-heap of size K: $O(N \log K)$. Sort then index: $O(N \log N)$. QuickSelect best on average — use randomized pivot.

Q9. Binary search in rotated sorted array?

→ One half is always sorted. Check if $\text{arr}[\text{mid}] > \text{arr}[\text{hi}]$ — left half sorted, else right half sorted. Then check if target falls in sorted half to decide which side to recurse. $O(\log N)$.

Dynamic Programming

Q10. How to approach a DP problem?

→ (1) Identify overlapping subproblems in recursive tree. (2) Define state — parameters that change between calls. (3) Write recurrence. (4) Implement: memoization top-down or tabulation bottom-up. (5) Optimize space if only previous row needed.

Q11. Longest Increasing Subsequence?

→ $O(N^2)$ DP: $dp[i] = \max(dp[j] + 1)$ for $j < i$ where $arr[j] < arr[i]$. $O(N \log N)$: patience sort — binary search in tails array for each element. Both should be known.

Q12. 0/1 Knapsack?

→ 2D DP: $dp[i][w] = \max$ value using first i items with capacity w . Include: $dp[i-1][w-weight] + value$. Exclude: $dp[i-1][w]$. $O(NW)$. Space optimize to 1D array traversed right to left.

Graphs

Q13. Detect cycle in directed graph?

→ DFS with 3 colors: white (unvisited), gray (in current DFS path), black (fully processed). Cycle if we hit a gray node. Alternatively Kahn's: if topological sort doesn't include all N nodes, a cycle exists.

Q14. Number of islands?

→ DFS or BFS from each unvisited '1'. Mark visited by setting to '0'. Count DFS/BFS invocations. $O(M \times N)$. Alternative: Union-Find for dynamic connectivity variant.

Q15. Dijkstra's algorithm?

→ Min-heap (dist, node). $dist[source] = 0$. Pop min, relax neighbors: if $dist[u] + weight < dist[v]$ update and push. Each node processed once. $O((V+E) \log V)$. Fails with negative edge weights — use Bellman-Ford instead.

General

Q16. Stack vs Queue — uses and Java implementation?

→ Stack (LIFO): balanced parentheses, undo, DFS, function call stack. Queue (FIFO): BFS, task scheduling. Java: use `ArrayDeque` for both. Stack: `push=addLast`, `pop=removeLast`. Queue: `offer=addLast`, `poll=removeFirst`.

Q17. HashMap vs TreeMap — when to use?

→ HashMap: $O(1)$ avg, no order. Frequency count, memoization, fast lookup. TreeMap: $O(\log N)$, sorted keys. Use when you need range queries (`subMap`, `headMap`), floor/ceiling, sorted iteration.

Q18. Merge K sorted lists?

→ Min-heap approach: add first element from each list. Pop min, add to result, push next from that list. $O(N \log K)$ where N =total elements, K =number of lists. Alternative: divide and conquer — merge pairs repeatedly, $O(N \log K)$.

Q19. Union-Find — what is it?

→ Disjoint Set structure for connected components. `find(x)`: returns root with path compression. `union(a,b)`: connect by rank. Both $O(\alpha(N))$ amortized — essentially $O(1)$. Use for: number of islands, Kruskal's MST, cycle detection.

Q20. Memoization vs tabulation?

→ Memoization (top-down): recursive + cache (HashMap or array). Natural recursion, only computes needed subproblems. Tabulation (bottom-up): iterative table from base case. No recursion overhead, cache-friendly, often faster in practice. Both give same complexity.